

Roll: 220121

Problem01:

Fifo page replacement algorithm:

Code:

```
#include <bits/stdc++.h>

using namespace std;

void marwan()
{
    cout << "Enter Number of processes:";

    int n;

    cin >> n;

    queue < pair<int, pair<int, int>> > q;

    for (int i = 0; i < n; i++)
    {
        cout << "Enter process arriving time and burst time";

        int a, b;

        cin >> a >> b;

        q.push({i, {a, b}});
    }

    while (!q.empty())
    {
        pair<int, pair<int, int>> p = q.front();

        q.pop();

        cout << "Process number being executed:" << p.first << endl;
    }

    cout << endl;
}

int32_t main()
{
    ios_base::sync_with_stdio(false);

    marwan();
}
```

```
    return 0;
}
```

Input: 4

2 3 4 5

Output:

Process Numbers Being Executed:2

Process Numbers Being Executed:3

Process Numbers Being Executed:4

Process Numbers Being Executed:5

Conclusion: Processes get executed in First in First out order using a Queue.

Problem02:

Bankers algorithm: Finding Safe Sequence of processes:

Code:

```
#include <bits/stdc++.h>

using namespace std;

void marwan()
{
    cout << "Enter Number of processes:";

    int n;

    cin >> n;

    cout << "Enter Number of Memories";

    int m;

    cin >> m;

    vector<vector<int>>> alloc(n, vector<int>(m)), mx(n, vector<int>(m)), need(n, vector<int>(m));

    cout << "Enter allocation matrix" << endl;

    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < m; j++)
        {
            cin >> alloc[i][j];
```

```

    }
}
cout << "Enter Max matrix" << endl;
for (int i = 0; i < n; i++)
{
    for (int j = 0; j < m; j++)
    {
        cin >> mx[i][j];
    }
}
for (int i = 0; i < n; i++)
{
    for (int j = 0; j < m; j++)
    {
        need[i][j] = mx[i][j] - alloc[i][j];
    }
}
vector<int> available(m);
cout << "Enter available memories:" << endl;
for (int i = 0; i < m; i++)
{
    cin >> available[i];
}
vector<int> vis(n + 1, 0), ss;
int count = 0;
while (count < n)
{
    int tcount = count;
    for (int i = 0; i < n; i++)
    {

```

```

    if (vis[i])
        continue;

    bool flag = true;
    for (int j = 0; j < m; j++)
    {
        if (need[i][j] > available[j])
        {
            flag = false;
        }
    }
    if (flag)
    {
        vis[i] = 1;
        ss.push_back(i);
        count++;
    }
}

if (count == tcount)
{
    cout << "Sequence Not Possible, page fault occurs." << endl;
    return;
}

cout << "Safe Sequence:";
for (auto vl : ss)
    cout << "p" << vl << " ";
cout << endl;
}

int32_t main()
{

```

```
ios_base::sync_with_stdio(false);  
marwan();  
return 0;  
}
```

Input: Given Matrix in the question.

Output: for the given matrix in the Question:

Safe Sequence: p0 p2 p3 p4 p2

Conclusion: A safe sequence is found or a page fault shows